

Consigne' exo 1

/ * C est une déclaration */

Procédure list (< list - param >)

1- Grammaire syntaxique

{ < Decl - procédure > $\rightarrow$ Procédure nom - proc (< list - param >)
< list - param > $\rightarrow$ param, < list - param > / }

2- Grammaire sémantique

{ < Decl - procédure > $\rightarrow$ Procédure nom - proc < A >
(< list - param >)
< list - param > $\rightarrow$ param, < B > < list - param > /
< A >, < B > $\rightarrow$ E param < B >

Routine Sémantique

Routine A $\rightarrow$ E

/ * Elle permet la recherche du nom de la procédure de la TS * /

debut

lookup (cd.nom, p)

Si p = 0 alors ('Err Sem : cd non existant

sinon / * préparer la ZD de la procéd
empiler (pile.données, display)

empiler (pile.données, le param. procéd

empiler (" , param. implicites

actr area $\leftarrow$ ZD procédure

nb. param $\leftarrow$ 0

fin

fin

Routine B $\rightarrow$ E / * Cherch param dans la TS * /

debut

lookup (cd.nom, p)

Si p = 0 alors ('Err Sem : inexistant')

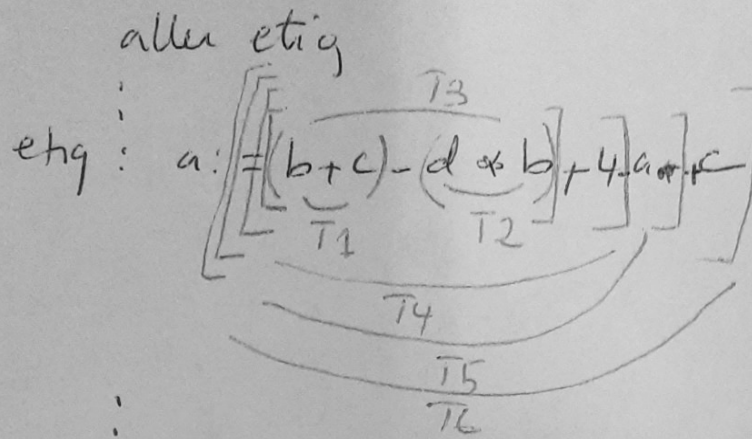
Remarque :
on peut le
2 routines

sinon param.type $\leftarrow$ p.type

empiler (pile.données param, param

nb. param $\leftarrow$ nb. param + 1

Exercice 2:



1. les quad nodes :

(BR, position, ,)

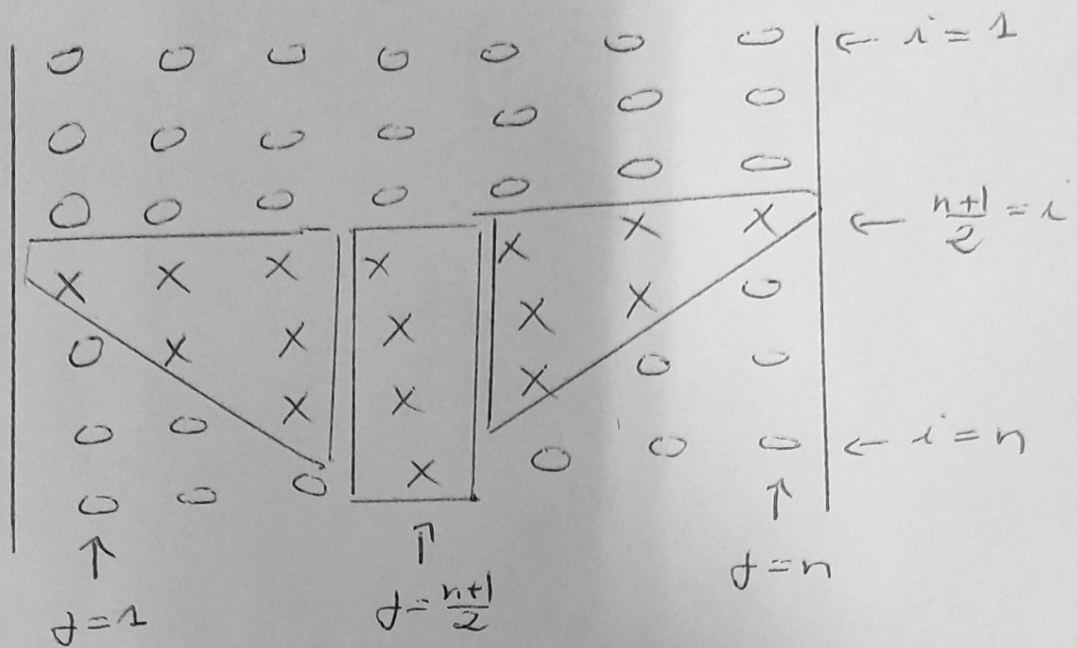
position : (+, b, c, T1)
(*, d, b, T2)
(-, T1, T2, T3)
(+, T3, 4, T4)
(-, T4, a, T5)
(+, T5, c, T6)
(:=, T6, , a)

2. code assembleur de la position :

Get in Acc	Acc	code
	side	
	side	JUMP position
Get in Acc ('b', 'c')	b	CHAR b
	T ₁	RAJ C
Get in Acc ('d', 'b')		AFFECT T ₁
	d	CHAR d
	T ₂	MULT b
Get in Acc (T ₂ , '1')		AFFECT T ₂
	T ₁	CHAR T ₁
	T ₃	SOUS T ₂
Get in Acc (T ₃ , '4')		RAJ 4
	T ₄	
Get in Acc (T ₄ , '1')	T ₅	SOUS a
Get in Acc (T ₅ , c)	T ₆	RAJ C
		AFFECT a

Exercice 3:

Pour $n=7$



- a. la représentation la plus optimale de la matrice η en mémoire est la représentation en mémoire dans une zone contiguë que les éléments non nuls de η .
- b. la relation liant i et j des éléments non nuls $\eta[i, j]$ de la matrice η :

$$\eta[i, j] \neq 0 \iff \left[\left(1 \leq j \leq \frac{n-1}{2} \right) \text{ and } \left(\frac{n+1}{2} \leq i \leq \frac{n+1}{2} + (j-1) \right) \right]$$

$$\text{or } \left[\left(j = \frac{n+1}{2} \right) \text{ and } \left(\frac{n+1}{2} \leq i \leq n \right) \right]$$

$$\text{or } \left[\left(\frac{n+3}{2} \leq j \leq n \right) \text{ and } \left(\frac{n+1}{2} \leq i \leq \frac{n+1}{2} + (n-j) \right) \right]$$

= car elle est vraie

- c. l'adresse de l'élément $\eta[i, j]$:

si and also si $(1 \leq j \leq \frac{n+1}{2})$ and $(\frac{n+1}{2} \leq i \leq \frac{n+1}{2} + (j-1))$

$$\underline{\text{also}} \quad \mathcal{D}[i, j] = \text{adBase} + \left(\sum_{k=2}^{j-1} k + \left(i - \frac{n+1}{2} \right) \right) \times \text{Taille}$$

si $(j = \frac{n+1}{2})$ and $(\frac{n+1}{2} \leq i \leq n)$

$$\underline{\text{also}} \quad \mathcal{D}[i, j] = \text{adBase} + \left(\sum_{k=2}^{j-1} k + \left(i - \frac{n+1}{2} \right) \right) \times \text{Taille}$$

2,5pts

$$\underline{\text{si}} \quad \mathcal{D}[i, j] = \text{adBase} + \left[\sum_{k=2}^{j-1} k + \frac{n+1}{2} + \sum_{k=\frac{n+3}{2}}^{n-j+1} (n-k+1) + \left(i - \frac{n+1}{2} \right) \right] \times \text{Taille}$$

si

si

$$\underline{\text{si}} \quad \mathcal{D}[i, j] = \text{nil}$$

si

d. 1. la méthode de représentation en utilisant le secteur bit:

$$\text{bit}(k) = \begin{cases} 0 & \text{si } \mathcal{D}[i, j] = 0 \\ 1 & \text{si } \mathcal{D}[i, j] \neq 0 \end{cases}$$

1pt

2. l'adresse d'un élément $\mathcal{D}[i, j]$ en utilisant le secteur bit:

Debut

$$K_i = (j-1) * n + i;$$

si $\text{bit}(K) = 0$ alors $DJ[i, j] = \text{nil}$

sinon $\text{compt} := 0;$

$l := 1;$

tant que $l < K$

faire si $\text{bit}(l) \neq 0$

alors $\text{compt} := \text{compt} + 1;$

fin;

$l := l + 1;$

fait;

$DJ[i, j] = \text{echBase} + \text{compt} * \text{Taille d'ell}$

Fin

Fin

1595